

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	S. Dhanashree	Reg. No.:	192325082
Section / Batch:	AIML	Date:	

Code of Conduct

I S. Dhanashree (192325082) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Dhanashree

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: john.doe123@gmail.com

Mobile: +91-9876543210

Password: P@ssw0rd123

Date of Birth: 15/08/2004

Register Number: 23AIML1056

Department: Artificial Intelligence and Machine Learning

For the given student details formulate Regex patterns for the following:

Email ID

Mobile Number

Strong Password

Date of Birth (DD/MM/YYYY)

Register Number

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.

Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.

Validation Criteria:

Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, \$, %, &, !, or *.
- Must not contain any whitespace characters.

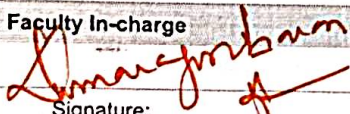
Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1	4	4	4	4	4	20	97
2	4	4	4	4	4	20	
3	4	4	4	4	4	20	
4	4	4	4	3	4	19	
5	4	4	4	3	4	19	

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: 22/7	Date: _____	Date: _____

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details

- Name: John Doe
- Email: john.doe123@gmail.com
- Mobile: +91-9876543210
- Password: P@ssw0rd123
- Date of Birth: 15/08/2004
- Register Number: 23AIML1056
- Department: Artificial Intelligence and Machine Learning

1. Email ID

Regex Pattern:

`^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`

2. Mobile Number

Regex Pattern:

`^\+91-\d{10}$`

3. Strong Password

Regex Pattern:

`^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#S%&!*])[A-Za-z\d@#S%&!*]{8,}$`

4. Date of Birth (DD/MM/YYYY)

Regex Pattern:

`^\d{2}/\d{2}/\d{4}$`

5. Register Number

Regex Pattern:

`^\d{2}[A-Z]{4}\d{4}$`

Section 2: Search for a String

Regex Pattern

`r'\bAI\b'`

Python Program

```
import re
```

```
text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
```

```
pattern = r'\bAI\b'
```

```
match = re.search(pattern, text)
```

```
if match:
```

```
    print("First occurrence found")
    print("Starting Position:", match.start())
    print("Ending Position:", match.end())
```

```
else:
```

```
    print("Word not found")
```

```
count = len(re.findall(pattern, text))
print("Total Occurrences:", count)
```

Intermediate Results

- First Match = AI
- Starting Position = 25
- Ending Position = 27
- Total Occurrences = 6

Output

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/admin/Documents/NLP Lab_Programs/Email.py
First occurrence found
Starting Position: 25
Ending Position: 27
Total Occurrences: 6
>>>
```

Section 3: Split the Documentation into Sentences and Words

Regex Patterns

Sentence Split

[.!?]+

Word Split

\s+

Python Program

```
import re
```

```
text = """Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand."""
```

```
sentences = [s.strip() for s in re.split(r'[.!?]+', text) if s.strip()]
words = re.split(r'\s+', text)
```

```
print("Total Sentences:", len(sentences))
print("Total Words:", len(words))
```

```
print("\nSentences:")
for s in sentences:
    print(s)
```

```
print("\nWords:")
print(words)
```

Intermediate Results

Extracted Sentences

1. Artificial Intelligence (AI) is transforming industries across the world
2. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences

3. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making
4. As AI continues to evolve, professionals with AI skills are in high demand

Total Sentences = 4

Total Words = 54

Output

```

C:\Users\admin\Documents\ML Lab Programs\Email.py
Total Sentences: 4
Total Words: 54

Sentence:
Artificial Intelligence (AI) is transforming industries across the world.
AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences.
Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making.
As AI continues to evolve, professionals with AI skills are in high demand.

Words:
['Artificial', 'Intelligence', 'AI', 'is', 'transforming', 'industries', 'across', 'the', 'world', 'AI', 'is', 'used', 'in', 'healthcare', 'to', 'assist', 'doctors', 'in', 'diagnosis',
 'in', 'banking', 'to', 'detect', 'fraud', 'and', 'in', 'education', 'to', 'provide', 'personalized', 'learning', 'experiences', 'Many', 'companies', 'invest', 'heavily', 'in',
 'AI', 'research', 'because', 'AI', 'improves', 'efficiency', 'and', 'enables', 'intelligent', 'decision-making', 'As', 'AI', 'continues', 'to', 'evolve', 'professionals', 'with', 'AI',
 'skills', 'are', 'in', 'high', 'demand']

```

Section 4: Email and Strong Password Validation

Email Validation

Regex Pattern

```
^[A-Za-z0-9]+[A-Za-z0-9._%+]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$
```

Password Validation

Regex Pattern

```
^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])\S{8,}$
```

Python Program

```
import re
```

```
email = "john.doe123@gmail.com"
```

```
password = "P@ssw0rd123"
```

```
email_pattern = r'^[A-Za-z0-9]+[A-Za-z0-9._%+]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
```

```
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])\S{8,}$'
```

```

if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

```

Intermediate Results

- Email satisfies all validation rules.
- Password contains:
 - Uppercase letter ✓
 - Lowercase letter ✓
 - Digit ✓
 - Special character ✓
 - Minimum 8 characters ✓
 - No whitespace ✓

Output

```

Valid Email
Strong Password
>> |

```

Section 5: Compile Once, Validate Many

Regex Pattern

```
^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$
```

Python Program

```

import re

emails = [
    "john@gmail.com",
    "student@college.edu",
    "abc@yahoo.org",
    "user123@test.in",
    "wrong_email@com"
]

```


1

```
pattern = re.compile(r'^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in|S)')
```

for email in emails:

```
    if pattern.fullmatch(email):
```

```
        print(email, "-> Valid")
```

```
    else:
```

```
        print(email, "-> Invalid")
```

Intermediate Results

Email ID	Validation
john@gmail.com	Valid
student@college.edu	Valid
abc@yahoo.org	Valid
user123@test.in	Valid
wrong_email@com	Invalid

Output

```
john@gmail.com -> Valid
student@college.edu -> Valid
abc@yahoo.org -> Valid
user123@test.in -> Valid
wrong_email@com -> Invalid
>>> |
```